



13. The Abstraction: Address Space & 15. Address Translation

Memory Virtualization

복습

Process

물리적으로는 어떻게 되냐

User Stack vs. Kernel Stack

Motivation for Virtualization

Multiprogramming Goals

Transparency (투명성)

Protection (보안)

Efficiency

Sharing

Abstraction: Address Space

Motivation for Dynamic Memory

Stack Organization

Where are Stacks Used?

Heap Organization

Quiz: Match that Address Location

Memory Accesses

Quiz: Memory Accesses?

How to Virtualize Memory?

1. Time Sharing of Memory

Problems with Time Sharing Memory

2. Static Relocation

Disadvantages

3. Dynamic Relocation

Hardware Support for Dynamic Relocation

Implementation of Dynamic Relocation: BASE REG

Dynamic Relocation with Base Register

Quiz: Who Controls the Base Register?

4. Dynamic with Base + Bounds

Implementation of BASE + BOUNDS

Managing Processes with Base and Bounds

Advantages

Disadvantages

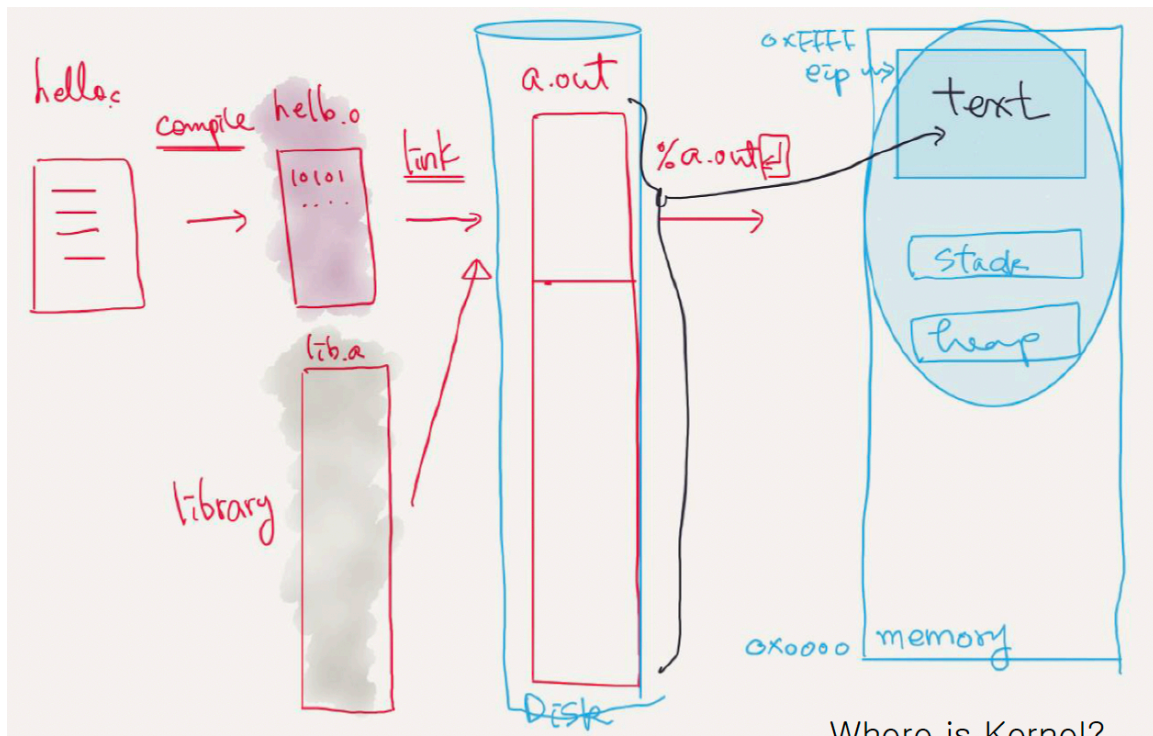
Memory Virtualization

- OS는 물리적 메모리를 가상화
- OS는 각 프로세스에게 illusion memory space 제공
- process 관점에서는 전체 memory를 독점해서 쓰는 것처럼 생각

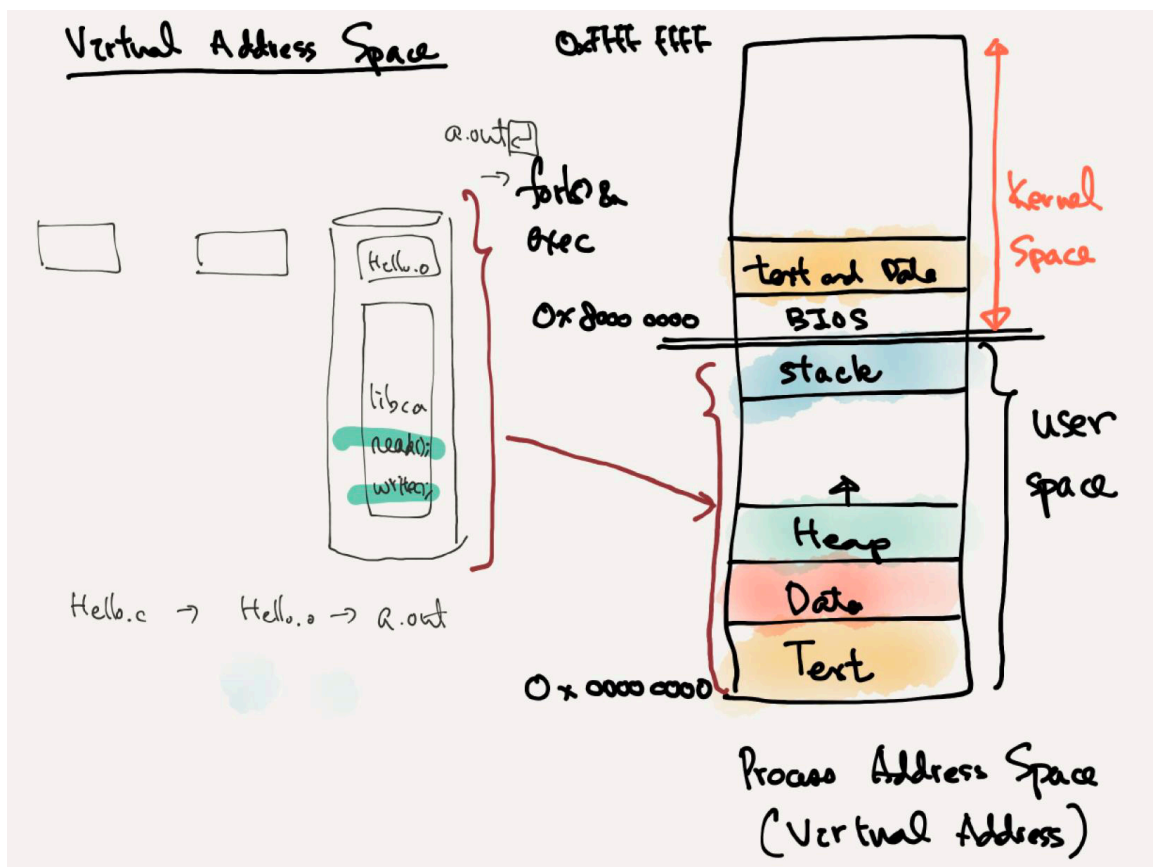
복습



- user stack: local variables, function call return address
- heap: dynamic allocated memory
- data: global and static variables
- text: instructions



- 그림 잘못됨 - text가 아래 위치해야 함



- 32bit라고 가정: 전체가 4GB

- 0 ~ 3GB: User space
- 3GB ~ 4GB: Kernel space

Process

```
struct thread{
    /* Owned by thread.c. */
    tid_t tid; /* Thread identifier. */
    enum thread_status status; /* Thread state. */
    char name[16]; /* Name (for debugging purposes). */
    uint8_t* stack; /* Saved stack pointer. */
    int priority; /* Priority. */
    struct list_elem allelem; /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem; /* List element. */

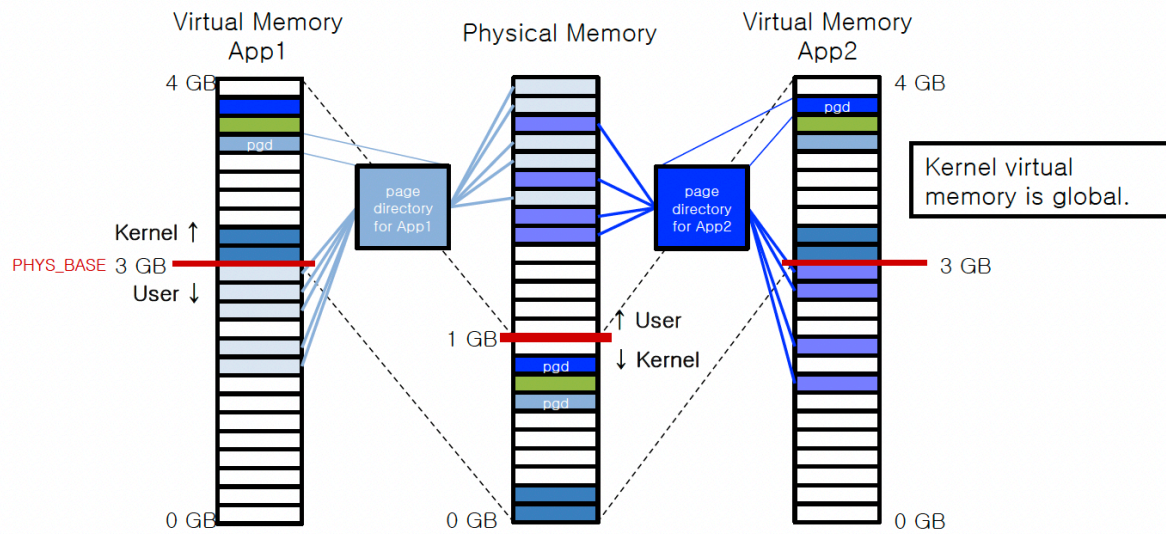
#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t* pagedir; /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic; /* Detects stack overflow. */
};
```

```
enum thread_status {
    THREAD_RUNNING,
    THREAD_READY,
    THREAD_BLOCKED,
    THREAD_DYING
};
```

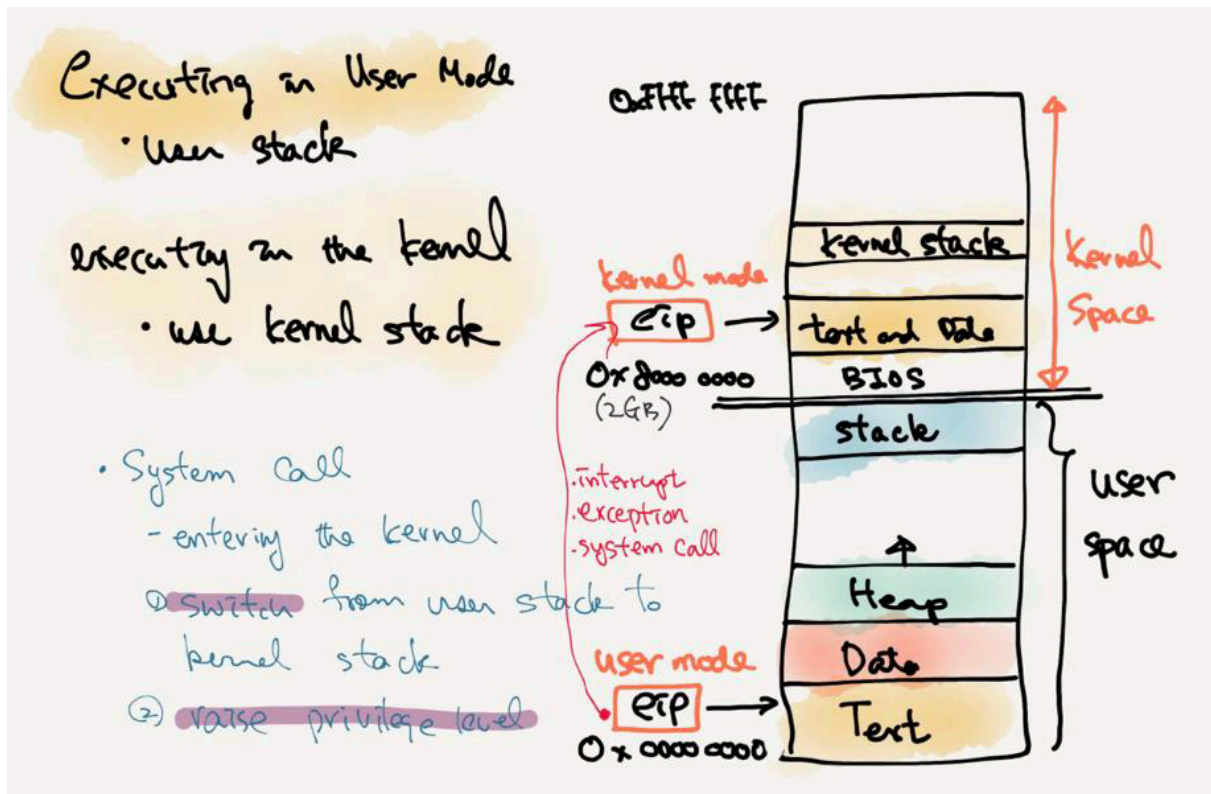
- `uint8_t *stack`은 kernel stack ptr
- `uint32_t *pagedir`은 가상의 주소를 물리적 주소로 변환하는 변환기 역할?

물리적으로는 어떻게 되냐



- 0부터 4GB면
 - 밑의 0부터 1GB까지는 Kernel - 1 : 1 mapping
 - 그러면 3GB 남은 건데 이걸 어떻게 process 2개가 sharing을 하냐 → 이걸 알아가자

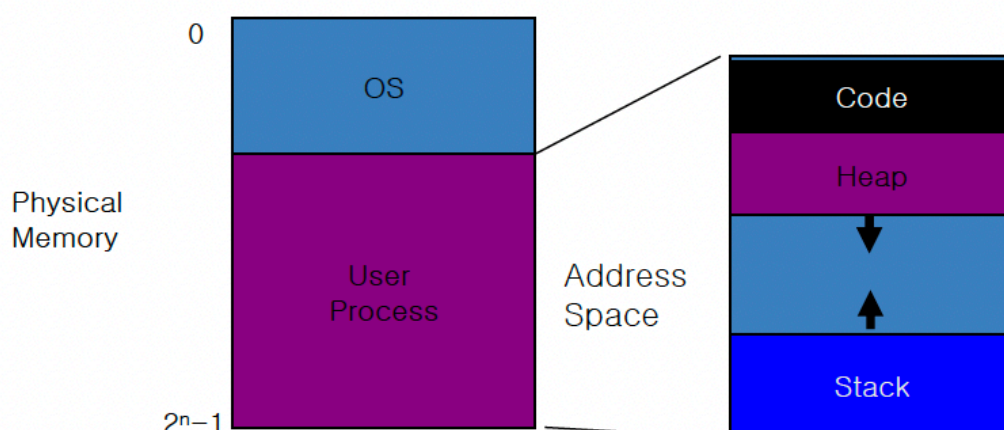
User Stack vs. Kernel Stack



- %eip는 가상 주소

Motivation for Virtualization

- **Uniprogramming:** One process runs at a time



- Disadvantages:
 - Only one process runs at a time

- Process can destroy OS
- 그림 잘못됨 - 오른쪽 뒤집힘

Multiprogramming Goals

Transparency (투명성)

- Processes are not aware that memory is shared
실행 중인 프로그램이 메모리가 가상화된다는 사실을 모르게 해야함.
- Works regardless of number and/or location of processes

Protection (보안)

- Cannot corrupt OS or other processes
- Privacy: Cannot read data of other processes

Efficiency

- Do not waste memory resources (minimize fragmentation)

Sharing

- Cooperating processes can share portions of address space

Abstraction: Address Space

- Address space: Each process has set of addresses that map to bytes
- Problem: How can OS provide illusion of private address space to each process?
- Review: What is in an address space?
- Address space has static and dynamic components
 - **Static:** Code and some global variables(Data)
 - **Dynamic:** Stack and Heap

Motivation for Dynamic Memory

- Why do processes need dynamic allocation of memory?
 - Do not know amount of memory needed at compile time
 - Must be pessimistic when allocate memory statically
 - Allocate enough for worst possible case; Storage is used inefficiently
- Recursive procedures
 - Do not know how many times procedure will be nested
- Complex data structures: lists and trees

```
struct my_t *p = (struct my_t*)malloc(sizeof(struct my_t));
```

- Two types of dynamic allocation
 - **Stack**
 - **Heap**

Stack Organization

- Definition: **Memory is freed in opposite order from allocation**
 - alloc(A); alloc(B); alloc(C); free(C); alloc(D); free(D); free(B); free(A);
- Simple and efficient implementation:
 - Pointer separates allocated and freed space
 - Allocate: Increment pointer
 - Free: Decrement pointer
- **No fragmentation**

Where are Stacks Used?

- OS uses stack for **procedure call frames (local variables and parameters)**

Heap Organization

- Definition: Allocate from any random location: `malloc()`, `new()`
 - Heap memory consists of allocated areas and free areas (holes)
 - Order of allocation and free is unpredictable
- Advantage
 - Works for all data structures
- Disadvantages
 - Allocation can be **slow**
 - End up with small chunks of free space → **fragmentation**
 - Where to allocate 12 bytes? 16 bytes? 24 bytes??
- What is OS's role in managing heap?
 - OS gives big chunk of free memory to process; library manages individual allocations

Quiz: Match that Address Location

```
int x;
int main(int argc, char* argv[]) {
    int y;
    int *z = malloc(sizeof(int));
}
```

Address	Location
x	static data
main	code
y	stack
z	stack
*z	heap

Memory Accesses

- **%rbp** is the base pointer: points to base of current stack frame

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char* argv[]) {
    int x;
    x = x + 3;
}
```

```
0x10: movl 0x8(%rbp), %edi
0x13: addl $0x3, %edi
0x19: movl %edi, 0x8(%rbp)
```

Quiz: Memory Accesses?

- Initial %rip = 0x10 %rbp = 0x200
- **%rbp** is the base pointer: points to base of current stack frame
%rip is instruction pointer (or program counter)
- Fetch instruction at addr 0x10
Exec: load from addr 0x208
- Fetch instruction at addr 0x13
Exec: no memory access
- Fetch instruction at addr 0x19
Exec: store to addr 0x208

How to Virtualize Memory?

- Problem: How to run multiple processes simultaneously?
 - Addresses are "hardcoded" into process binaries
 - 위에서 본 예시: 0x10, 0x13, 0x19
- How to avoid collisions?
- Possible Solutions for Mechanisms
 1. Time Sharing
 2. Static Relocation
 3. Base

4. Base + Bounds

1. Time Sharing of Memory

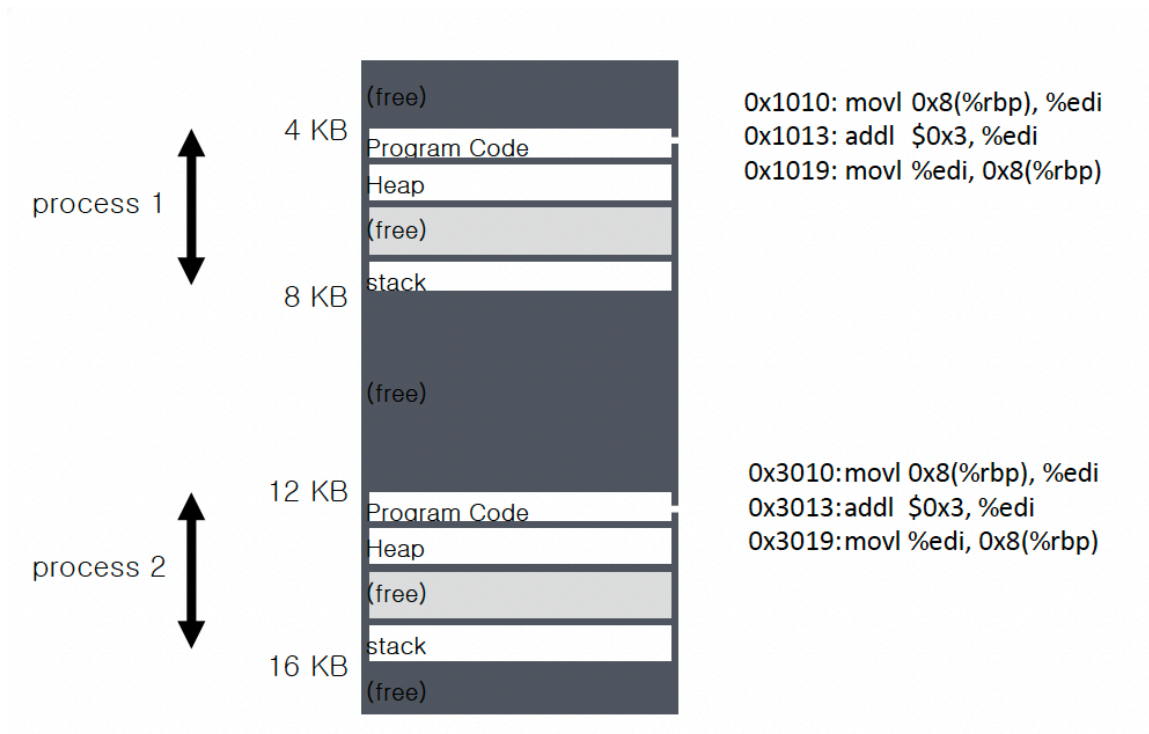
- Try similar approach to how OS virtualizes CPU
- Observation:
 - OS gives illusion of many virtual CPUs by saving **CPU registers** to **memory** when a process isn't running
- Could give illusion of many virtual memories by saving **memory** to **disk** when process isn't running

Problems with Time Sharing Memory

- Ridiculously poor performance
- Better Alternative: **space sharing**
 - At same time, space of memory is divided across processes
- **Remainder of solutions all use space sharing**

2. Static Relocation

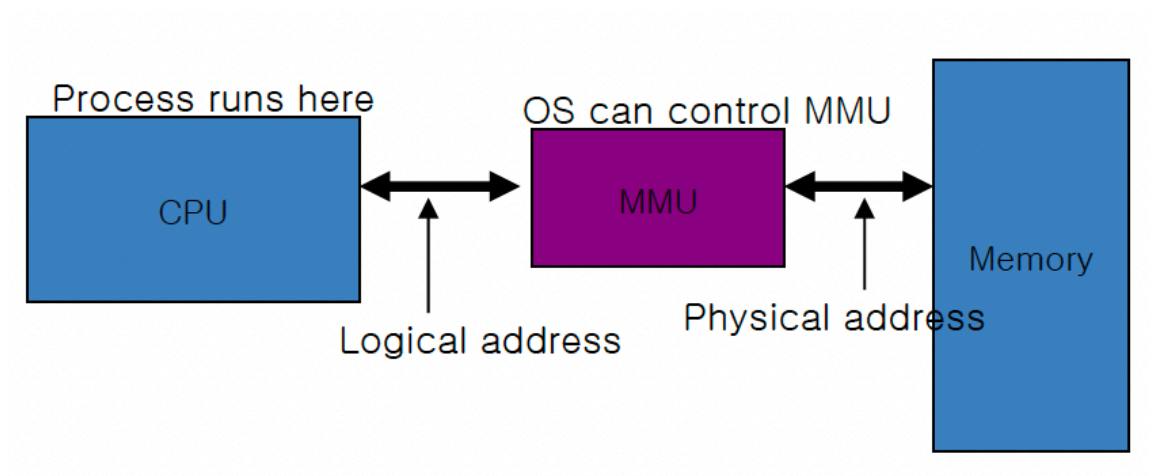
- Idea: OS rewrites each program before loading it as a process in memory
memory에 process를 로드하기전에 rewrite한다
- Each rewrite for different process uses different addresses and pointers
- Change jumps, loads of static data



Disadvantages

- **No protection**
 - **Process can destroy OS or other processes**
 - **No privacy**
- **Cannot move address space after it has been placed**

3. Dynamic Relocation



- Goal: **Protect** processes from one another
- **hardware support** 필요함
 - **Memory Management Unit (MMU)**
- MMU dynamically changes process address at every memory reference
프로세스가 메모리에 접근할 때마다 MMU가 실시간으로 주소를 변환
 - Process generates **logical or virtual addresses** (in their address space)
 - Memory hardware uses **physical or real addresses**

Hardware Support for Dynamic Relocation

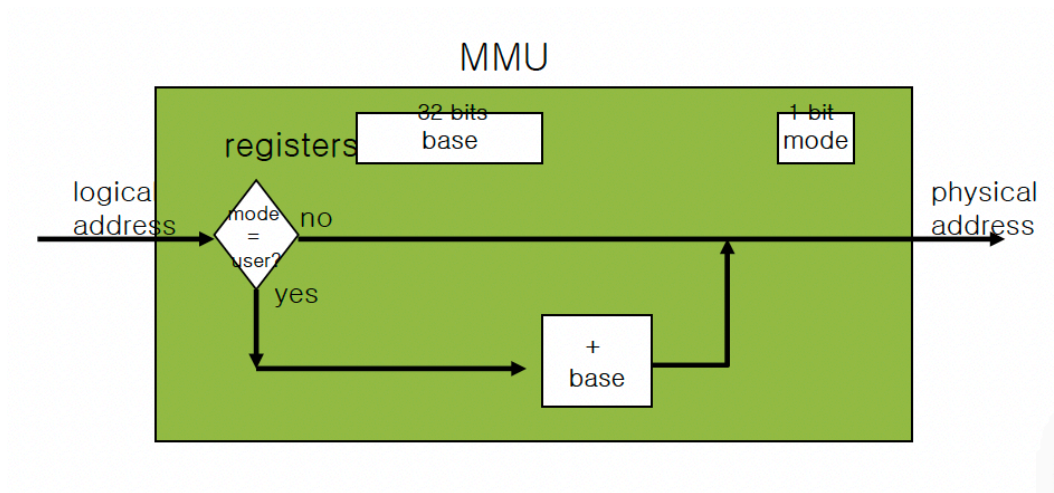
Two operating modes

- **Privileged (protected, kernel) mode**: OS runs
 - When enter OS (**trap, system calls, interrupts, exceptions**)
 - Allows certain instructions to be executed
 - **Can manipulate contents of MMU!!**
 - **Allows OS to access all of physical memory!!**
- **User mode**: User processes run
 - **Perform translation of logical address to physical address**

Minimal MMU contains **base register** for translation

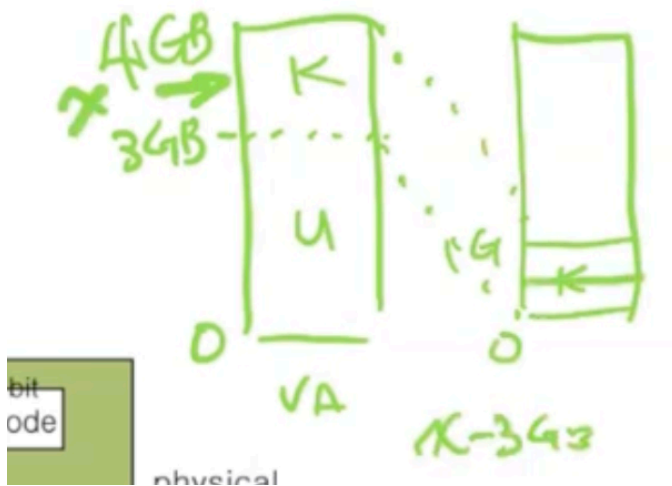
- base: start location for address space
 - base값은 process context에 저장?

Implementation of Dynamic Relocation: BASE REG



Translation on every memory access of user process

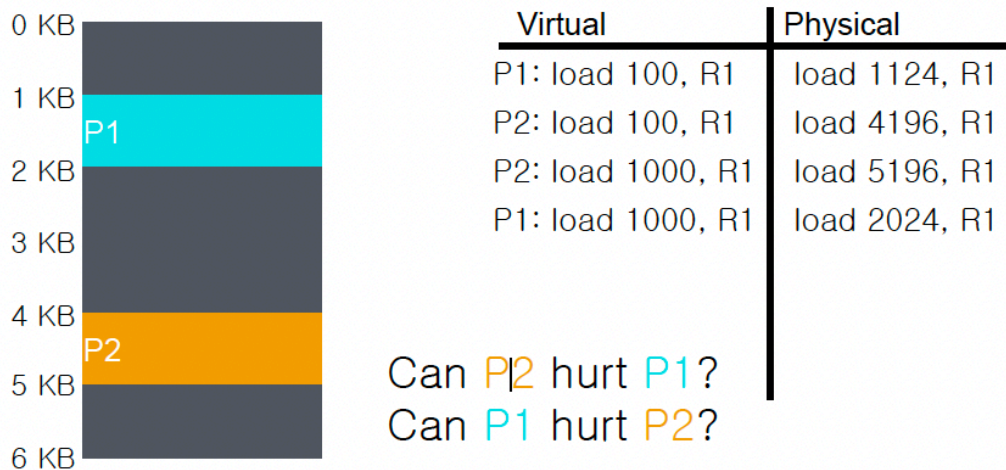
- MMU adds base register to logical address to form physical address



- 좌측이 virtual, 우측이 physical
 - kernel 영역의 경우 3GB만큼 빼주면 됨 (1:1 매핑)

Dynamic Relocation with Base Register

- Idea: translate virtual addresses to physical by adding a fixed offset each time.
- Store offset in base register
- Each process has different value in base register



Quiz: Who Controls the Base Register?

- What entity should **do translation of addresses with base register**?
 - (1) process (2) OS **(3) HW - user mode에서 일어나긴 함**
- What entity should **modify** the base register?
 - (1) process **(2) OS** (3) HW

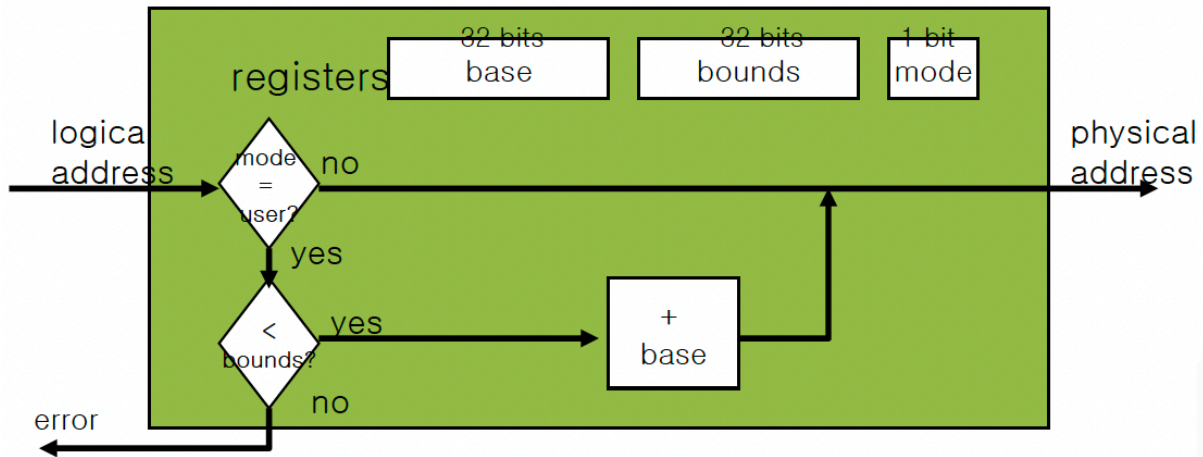
base만 쓰면 boundary check를 안해서 침범할수도 있다

- 위로 가거나 아래로 가거나

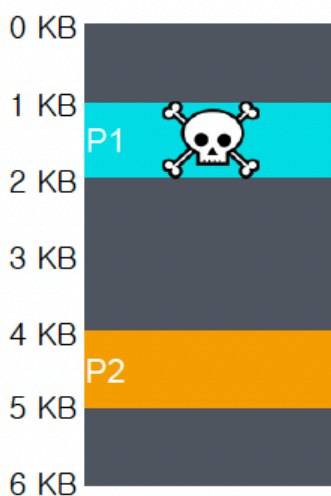
4. Dynamic with Base + Bounds

- Idea: limit the address space with a bounds register
- Base register:** **smallest** physical addr(or starting location)
- Bounds register:** **size** of this process's virtual address space
 - Sometimes defined as largest physical address (base + size)
- OS kills process if process loads / stores beyond bounds

Implementation of BASE + BOUNDS



- Translation on every memory access of user process
 - MMU compares logical address to bounds register
 - if logical address is greater, then generate error
 - MMU adds base register to logical address to form physical address



Virtual	Physical
P1: load 100, R1	load 1124, R1
P2: load 100, R1	load 4196, R1
P2: load 1000, R1	load 5196, R1
P1: load 100, R1	load 2024, R1
P1: store 3072, R1	interrupt OS!

Can P1 hurt P2?

3072에 R1값을 집어넣어라 → $3072 > 1024$ → 에러

Managing Processes with Base and Bounds

- **Context-switch**
 - Add base and bounds registers to PCB

- Steps
 1. **Change to privileged mode**
 2. **Save base and bounds registers of old process**
 3. **Load base and bounds registers of new process**
 4. **Change to user mode and jump to new process**
- What if don't change base and bounds registers when switch? → 큰 일 남
- Protection requirement
 - **User process cannot change base and bounds registers**
user 프로세스는 base와 bound 레지스터 값을 변경할 수 없다
 - **User process cannot change to privileged mode**
user 프로세스는 스스로 privileged mode로 변경할 수 없다

Advantages

- Provides protection (both read and write) across address spaces
보호를 제공한다
- Supports **dynamic relocation**
 - Can place process at different locations initially and **also move address spaces**
- Simple, inexpensive implementation
간단하다! 저렴하다!
 - Few registers, little logic in MMU
- Fast
 - Add and compare in parallel

Disadvantages

- Each process must be **allocated contiguously** in physical memory
 - Must allocate memory that may not be used by process
- No partial sharing: Cannot share limited parts of address space

segmentation에 대해 개념적으로 알고 있어라?

레지스터가 더 추가됨 6개?